Why `constant_wrapper` is not a usable replacement for `nontype`

Document #:	P3792R0 [Latest] [Status]
Date:	2025-07-13
Project:	Programming Language C++
Audience:	LEWG, LWG

Reply-to:	Bronek Kozicki <brok@incorrekt.com>

Contents

- 1 Abstract
- 2 Polls taken
- 3 Discussion
  - 3.1 Why does `function_ref` take a `nontype` parameter ?
  - 3.2 Would other function wrappers benefit from taking a `nontype` wrapper ?
  - 3.3 Is there a proposal to add such constructors to other function wrappers ?
  - 3.4 What's the problem with replacing `nontype` with `constant_wrapper` ?
  - 3.5 It worked, so what's the problem ?
  - 3.6 Example user code breakage
  - 3.7 What about change in `invoke` and `invoke_r` ?
- 4 Summary
- 5 Future
- 6 Acknowledgments
- 7 References

§ 1 Abstract

The lack of LEWG consensus during the Sofia meeting to replace `nontype` with `constant_wrapper` as a `function_ref` construction parameter has reportedly surprised some members of LWG. Plausibly, it might have also surprised the wider C++ community. This paper is an attempt to explain the process and the rationale behind the (lack of) decision for such a change.

§ 2 Polls taken

During the discussion on [P3740R1] in Sofia, LEWG took a series of polls which ultimately led to the decision to keep `nontype` parameters in `function_ref` constructors and rename it, rather than replace `nontype` with `constant_wrapper`, while opening a path for a possible NB comment which might request such a change.

First, the following three polls were taken as popularity contest:

> **POLL:** Forward "P3740R1: Last chance to fix `std::nontype`" selecting Option A (change from `nontype` to `constant_wrapper`, also add overloads to other function wrappers (`std::function`)) to LWG

SF	F	N	A	SA
0	4	0	7	8

> Outcome: No consensus for change

> **POLL:** Forward "P3740R1: Last chance to fix `std::nontype`" selecting Option A (change from `nontype` to `constant_wrapper`, DO NOT add overloads to other function wrappers (`std::function`)) to LWG

SF	F	N	A	SA
6	7	2	6	0

> Outcome: Weak consensus in favor

> **POLL:** Forward "P3740R1: Last chance to fix `std::nontype`" selecting Option B (rename `std::nontype` to `std::constant_arg`) to LWG

SF	F	N	A	SA
7	9	2	2	1

> Outcome: Consensus in favour

As a result of the popularity contest, the Option B was assumed to have become the "status quo", and the two options with (some) consensus have been pitted against each other in the final poll:

> **POLL:** Forward "P3740R1: Last chance to fix `std::nontype`" selecting Option A (change from `nontype` to `constant_wrapper`, DO NOT add overloads to other function wrappers (`std::function`)) instead of Option B (rename `std::nontype` to `std::constant_arg`) to LWG

SF	F	N	A	SA
9	4	2	4	4

> Outcome: No consensus for change.

The final notes from the meeting do recognize that Option B has received fewer votes than the limited Option A:

> The second poll had no consensus, but got more votes toward the more extensive change (remove `nontype` and replace with `constant_wrapper` in `function_ref` only). This will need to be discussed again if an NB comment is submitted for C++26.

§ 3 Discussion

During the discussion leading to the polls, I have argued strongly against replacing `nontype` with `constant_wrapper` as a parameter for `function_ref` construtors, instead arguing for renaming `nontype` to something that better reflects its use as a `function_ref` construction parameter (which is the only use of `nontype` in the standard).

This was informed by my implementation experience [zhihaoy/nontype_functional/pull/13] replacing `nontype` with `constant_wrapper` in reference implementations of `function_ref`, `move_only_function` and `function`. The problem was not related to the implementation (that was the easy part). It was the result of a discovery of inconsistencies in the standard, which would potentially lead the users to write error-prone C++ code, had such change been made.

§ 3.1 Why does `function_ref` take a `nontype` parameter ?

The details can be found in [P2472R3], but the short version is that it provides `function_ref` with a type-erasing constructor from a member function or a free function. The mechanics of this constructor is simple:

- initialize the exposition-only `thunk-ptr` with the address of a function which wraps the `invoke_r` call taking the value template parameter of `nontype` as a first parameter, and optionally second parameter to be dereferenced from `bound-entity`
- initialize the exposition-only `bound-entity` with the optional second constructor parameter (if it is a pointer) or its address (if it is a reference)

This allows the `function_ref` to, for example, wrap a pointer to a member function *and* the object reference to call it with.

§ 3.2 Would other function wrappers benefit from taking a `nontype` wrapper ?

Potentially. In particular `move_only_function` and `copyable_function`, which both apply small size optimization, might be able to use space more efficiently, if they performed

type-erasure from a `nontype` wrapper (similarly like `function_ref` does) when instantiating the wrapper to call target.

However, a similar optimization opportunity can be also created without the use of `nontype`. A function wrapper might recognize that an invocable passed to its constructor is stateless (e.g. by application of `is_empty` and `is_trivially_relocatable` traits) and then simply do not waste space trying to store such non-state.

§ 3.3 Is there a proposal to add such constructors to other function wrappers ?

Initial work has been done by Zhihao Yuan and (separately) by Tomasz Kamiński.

§ 3.4 What's the problem with replacing `nontype` with `constant_wrapper` ?

In short, `constant_wrapper` has its own `operator()`, with its own semantics which might, or might not, be compatible with an arbitrary invocable used to instantiate it.

More specifically, `operator()` in `constant_wrapper` is defined in [P2781R8] as:

```
template<constexpr-param T, constexpr-param... Args>
  constexpr auto operator()(this T, Args...) noexcept
   requires requires(Args...) { constant_wrapper<T::value(Args::value...)>
        (); }
    { return constant_wrapper<T::value(Args::value...)>{}; }
```

Please note that the return type is an instance of `constant_wrapper`, which will fail compilation if the return type returned from the invocation `T::value(Args::value...)` is *not* a structural type. Also, the call parameters are all expected to have `value` static data member, which is extracted inside the call. There is nothing wrong with this, because the design goal of `constant_wrapper` is to provide C++ users with a *more useful* compile-time constant (compared to `integral_constant`). However it also means that an invocable returning an *arbitrary* (including non-structural) type *cannot* be used with `constant_wrapper::operator()`. This is by design, and (in my humble opinion) it's *good*.

When trying to use `constant_wrapper` in place of `nontype`, the workaround for this limitation is obvious: do not try to invoke the `constant_wrapper`, just unwrap its template parameter using `value` static data member. This is what I did and it worked.

§ 3.5 It worked, so what's the problem ?

Currently no other function wrapper has this functionality. However, the C++26 status quo is that `constant_wrapper`, instantiated with a value of a compatible type, is also an invocable, and it can be used to construct any function wrapper (with matching template parameters). The behaviour and semantics of such function wrapper will be different compared to a `function_ref` constructed with `constant_wrapper`, because of the semantics of `con-`

stant_wrapper::operator(). This difference will be most stark if the invocable used to instantiate constant_wrapper is a functor user type with overloaded function call operators, or with a templated function call operator (as a niebloid might be). In short, it is not safe to *just* substitute nontype with a constant_wrapper.

This means that:

— if nontype was replaced by constant_wrapper as a construction parameter to function_ref *and*

— a user performed a (seemingly) simple refactoring by swapping a standard function wrapper with another standard function wrapper (e.g. to benefit from the low cost of function_ref or to use the data storage in other wrappers) where the constructor happens to rely on constant_wrapper

… any of the following might happen:

— the program will continue to work as designed

— the program will fail to compile

— the program will continue to compile and "work", but with subtly changed behaviour

In order to prevent the last two happening, we would have to do some of the following:

— revisit operator() in constant_wrapper i.e. [P2781R8]

— revisit nontype parameter in function_ref i.e. [P2472R3] and [P0792R14]

— add similar overloads to other functional wrappers (see first poll taken)

— define specialization of invoke and invoke_r for constant_wrapper to use the extracted value rather than call operator() directly

§ 3.6 Example user code breakage

In the following code excerpt, foo_t is a niebloid providing two different overloads of operator(), selected by means of a requires clause.

```
static constexpr struct foo_t final
{
    constexpr auto operator()(auto &&...args) const -> int
            requires(std::integral<std::remove_cvref_t<decltype(args)>> &&
        ...)
    {
        return (0 + ... + args);
    }

    constexpr auto operator()(auto &&...args) const -> int
        requires(std::integral<
                        decltype(std::remove_cvref_t<decltype(args)>::val-
        ue)> &&
                 ...)
    {
```

```
        return sizeof...(args);
    }

} foo = {};
```

The second overload matches types providing a `value` static data member, just like `constant_wrapper` or a `baz_t` type presented below:

```
static constexpr struct baz_t final
{
    static constexpr int value = 42;
} baz = {};
```

The `foo` object is used to instantiate a `cw<foo>`, from which a `move_only_function<int(baz_t)> fn` is constructed. This works because (by design) `constant_wrapper::operator()` will accept any parameter type with a `value` static data member, including `baz_t` presented above.

```
auto main() -> int
{
    move_only_function<int(baz_t)> fn(cw<foo>);
    assert(fn(baz) == 42);
}
```

The `assert` will succeed if the top overloaded `operator()` in `foo_t` is selected. This is guaranteed by `constant_wrapper::operator()`, which unwraps `value` from `baz_t`, *before* submitting the result to `foo(int)`.

If the user were to switch their code from `move_only_function` to `function_ref`, taking `constant_wrapper` as a constructor parameter (in place of the current `nontype`), then the constructor will unwrap `value` (that is, `foo` object) from `cw<foo>` and invoke `foo(baz_t)` (rather than call `constant_wrapper::operator()`). As a result, the second overload of `operator()` in `foo_t` will be selected, and the assert will fail:

```
auto main() -> int
{
    function_ref<int(baz_t)> fn(cw<foo>);
    assert(fn(baz) == 42); // assertion failure
}
```

This demonstration is available on github [Bronek/nontype_functional/pull/1].

§ 3.7 What about change in `invoke` and `invoke_r` ?

The last option suggested above, which nobody is proposing and which (in my opinion) is *not* sensible, would "fix" the problem, preventing breakage when the user code is switched from one standard function wrapper to another (when the constructor call relies on on `constant_wrapper` parameter), at the cost of imbuing the `constant_wrapper` with *dual* invocation semantics:

— direct call defined in `operator()`, as stated in [P2781R8] *and*

— call into the value template parameter, via `value` static data member, with `invoke` and `invoke_r`

These dual semantics mean that we have moved the problem from one place to another. A user would not expect that a change in their code from `invoke` (or `invoke_r`) to function call syntax, might make the program fail to compile or worse, change its behaviour.

This is just making things worse.

§ 4 Summary

LEWG did the right thing in Sofia by not replacing `nontype` constructor parameters in `function_ref` with `constant_wrapper`. Had this been done, we would have to either revisit several other design decision taken previously or risk users' code breaking (sometimes subtly) when they move between different standard function wrappers. Even if we changed the other function wrappers to do what `function_ref` does (a design choice rejected by LEWG) then the inconsistency would remain, it would just move elsewhere.

§ 5 Future

I suggest we should pursue the following:

— Rename `nontype` to better reflect its current use in `function_ref` constructor. This *has* to be done in C++26 (possibly via NB comment) and is discussed in [P3774R0].

— Extend such newly named type with function call semantics that does not break user expectations, while preserving its lack of state. This is also discussed in [P3774R0].

— Consider optimizing other function wrappers for such a stateless constructor parameter, possibly as QoI.

§ 6 Acknowledgments

I am grateful to Jan Schultke, Tomasz Kamiński and Gašper Ažman for their kind comments and suggestions. Also big thank you to Zhihao Yuan and Zach Laine for their reference implementations of function wrappers and `constant_wrapper`, used when writing this paper.

§ 7 References

[Bronek/nontype_functional/pull/1] Bronek Kozicki. Breakage demonstration.
https://github.com/Bronek/nontype_functional/pull/1

[P0792R14] Vittorio Romeo, Zhihao Yuan, Jarrad Waterloo. 2023-02-09. function_ref: a non-owning reference to a Callable.
https://wg21.link/p0792r14

[P2472R3] Jarrad J. Waterloo, Zhihao Yuan. 2022-05-13. make function_ref more functional.
https://wg21.link/p2472r3
[P2781R8] Zach Laine, Matthias Kretz, Hana Dusíková. 2025-03-16. std::constexpr_wrapper.
https://wg21.link/p2781r8
[P3740R1] Jan Schultke. Last chance to fix std::nontype.
https://isocpp.org/files/papers/P3740R1.html
[P3774R0] Jan Schultke. Rename std::nontype, and make it broadly useful.
https://isocpp.org/files/papers/P3774R0.html
[zhihaoy/nontype_functional/pull/13] Bronek Kozicki. Replace nontype with constant_wrapper as a construction parameter.
https://github.com/zhihaoy/nontype_functional/pull/13